

Fondation
UCAD



UNIVERSITE
CHEIKH ANTA DIOP
DE DAKAR (UCAD)

ECOLE SUPERIEURE POLYTECHNIQUE



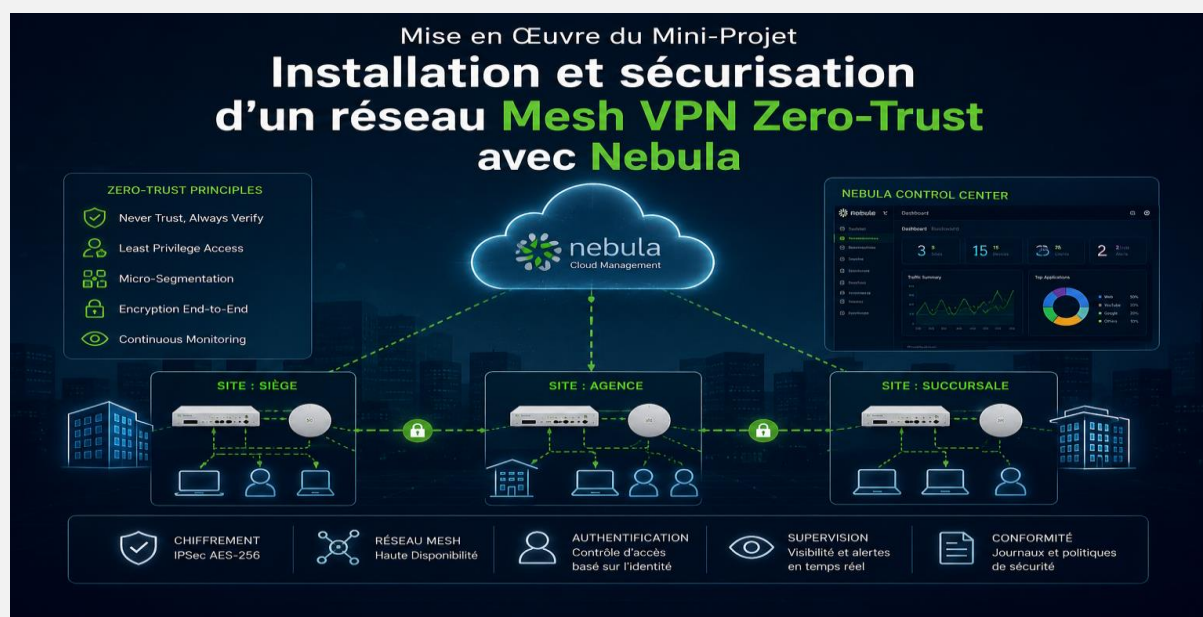
Département Génie Informatique

Licence 3 en Systèmes Réseaux et Télécommunications

Technologie de Sécurité (No-de-Sec)

Etudiant :

El Hadji Mamadou Ndiaye



Mise en Œuvre du Mini-Projet : Installation et sécurisation d'un réseau Mesh VPN Zero-Trust avec Nebula

▣ Durcissement (hardening) d'un serveur (notre Kali)

```
(elmadou@elhadjimamadoundiaye)-[~]
$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)

(elmadou@elhadjimamadoundiaye)-[~]
$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)

(elmadou@elhadjimamadoundiaye)-[~]
$ sudo ufw allow ssh
Rules updated
Rules updated (v6)

(elmadou@elhadjimamadoundiaye)-[~]
$ sudo ufw allow 8080
Rules updated
Rules updated (v6)

(elmadou@elhadjimamadoundiaye)-[~]
$
```

Nous configurons les politiques par défaut de **UFW** (Uncomplicated Firewall) sur Kali Linux en appliquant une approche restrictive de type « Deny All » pour tout le trafic entrant. Cette sécurisation est ensuite affinée par l'ouverture sélective des ports nécessaires, notamment le **port 22 (SSH)** pour l'administration à distance et le **port 8080** pour un service spécifique. Cette démarche illustre parfaitement le principe du **moindre privilège**, garantissant que seuls les flux explicitement autorisés peuvent atteindre le système.

```
(elmadou@elhadjimamadoundiaye)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
8080 ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
8080 (v6) ALLOW IN Anywhere (v6)

(elmadou@elhadjimamadoundiaye)-[~]
$
```

La commande `sudo ufw status verbose` confirme que le pare-feu est **actif** et applique une politique de sécurité rigoureuse en rejetant par défaut toutes les connexions entrantes. Seules les exceptions pour les ports **22/tcp (SSH)** et **8080** ont été validées, permettant un accès contrôlé tout en protégeant les autres services du système. Cette configuration garantit que la machine reste invisible et sécurisée face aux tentatives d'accès non autorisées sur le reste de la plage réseau.

Étape 2 : Sécurité Avancée avec SELinux

Cela ajoute un contrôle très strict sur ce que les programmes ont le droit de faire.

```
(elmadou@elhadjimamadoundiaye)-[~]
$ sudo lynis audit system

[ Lynis 3.1.6 ]

#####
###
Lynis comes with ABSOLUTELY NO WARRANTY. This is free software, and you are
welcome to redistribute it under the terms of the GNU General Public License.
See the LICENSE file for details about using this software.

2007-2025, CISofy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)
#####
###

[+] Initializing program
-----
- Detecting OS ... [ DONE ]
- Checking profiles ... [ DONE ]
- Detecting language and localization [ fr ]

Que faire:
Please submit a pull request on GitHub to include your OS version and the end
date of this OS version is being supported
URL: https://github.com/CISofy/lynis

=====
===

Lynis 3.1.6

Auditing, system hardening, and compliance for UNIX-based systems
(Linux, macOS, BSD, and others)

2007-2025, CISofy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)

=====
===

[TIP]: Enhance Lynis audits by adding your settings to custom.prfl (see /etc
/lynis/default.prfl for all settings)

(elmadou@elhadjimamadoundiaye)-[~]
$
```

Nous effectuons un audit de sécurité approfondi de son système Linux à l'aide de l'outil open

source **Lynis**, comme le montre la commande `sudo lynis audit system`. Ce processus permet de scanner la configuration, les logiciels installés et les paramètres réseau afin de détecter d'éventuelles vulnérabilités ou mauvaises configurations. Parallèlement, l'interface de **RustDesk** révèle une tentative de connexion vers une IP spécifique, soulignant l'importance d'un durcissement système avant d'autoriser des flux de contrôle à distance. Cette approche combinée de monitoring et d'audit est essentielle pour garantir la conformité et la résilience d'un poste d'administration dans un environnement **Zero-Trust**.

1. Explication du système

Nebula est un réseau superposé (overlay network) de type "Mesh VPN" conçu pour relier des hôtes situés derrière des pare-feu ou des routeurs NAT, sans nécessiter de configuration complexe de redirection de ports. Contrairement aux VPN traditionnels basés sur une architecture client-serveur rigide, Nebula permet à chaque nœud de communiquer directement avec les autres en utilisant le protocole de cryptage Noise, garantissant une latence minimale et une sécurité maximale.

Le système repose sur trois composants clés : une **Autorité de Certification (CA)** privée qui signe les clés de chaque machine, des **Lighthouses** (phares) qui servent d'annuaires pour que les nœuds se trouvent mutuellement, et des **Nœuds** qui établissent des tunnels cryptés entre eux. Cette architecture permet de créer un périmètre de sécurité "Zero-Trust" où seules les machines possédant un certificat valide peuvent voir ou interroger les services du réseau privé.

2. Documentation étape par étape

Étape 1 : Installation des binaires

Puisqu'on est sur Kali, nous allons récupérer les binaires officiels (plus propre que `apt` pour ce projet).


```
nebula-linux-amd64. 100%[=====>] 12,18M 371KB/s ds 33s
2026-04-16 01:11:12 (374 KB/s) - « nebula-linux-amd64.tar.gz » sauvegardé [12
769815/12769815]

(elmamadou@elhadjimamadoundiaye) - [~/nebula-project]
$ tar -xzf nebula-linux-amd64.tar.gz

(elmamadou@elhadjimamadoundiaye) - [~/nebula-project]
$ sudo mv nebula nebula-cert /usr/local/bin/
[sudo] Mot de passe de elmamadou :

(elmamadou@elhadjimamadoundiaye) - [~/nebula-project]
$
```

Nous procédons à l'installation des composants binaires de **Nebula** sur Kali Linux en extrayant l'archive téléchargée et en déplaçant les exécutables vers le répertoire système `/usr/local/bin` pour un accès global. Parallèlement, l'interface **RustDesk** montre une tentative de connexion infructueuse vers l'IP `192.168.100.9(windows11)`, illustrant la nécessité de configurer correctement le tunnel VPN avant d'établir le flux de contrôle. Cette étape de préparation logicielle est le socle indispensable pour déployer l'architecture de réseau maillé et sécuriser les communications futures entre les nœuds.

Étape 2 : Création de l'Autorité de Certification (CA)

C'est la base de ma sécurité. **À ne faire qu'une seule fois sur ma machine de gestion (mon Kali).**

```
(elmamadou@elhadjimamadoundiaye) - [~/nebula-project]
$ nebula-cert ca -name "ESP-Dakar-Net"

(elmamadou@elhadjimamadoundiaye) - [~/nebula-project]
$ ls
ca.crt  ca.key  nebula-linux-amd64.tar.gz

(elmamadou@elhadjimamadoundiaye) - [~/nebula-project]
$
```

Nous initialisons sa propre Autorité de Certification (CA) nommée « ESP-Dakar-Net » à l'aide de l'outil `nebula-cert`, générant ainsi les fichiers essentiels `ca.crt` et `ca.key`. Cette étape cruciale de la PKI (Public Key Infrastructure) permet d'établir la racine de confiance nécessaire pour signer et authentifier tous les futurs nœuds du réseau Mesh Nebula.

Générer le certificat racine et la clé privée du CA

"ESP-Dakar-Net" est le nom de mon réseau.

On a réussi à créer deux fichiers : `ca.crt` (à partager) et `ca.key` (**À GARDER SECRET**).

Étape 3 : Génération des certificats pour les nœuds

Imaginons deux nœuds : mon **Lighthouse** (Serveur central) et mon **L3-Student** (mon Kali).

```
(elmadou@elhadjimamadoundiaye)-[~/nebula-project]
$ nebula-cert sign -name "lighthouse" -ip "192.168.100.1/24"

(elmadou@elhadjimamadoundiaye)-[~/nebula-project]
$ nebula-cert sign -name "L3-Student" -ip "192.168.100.2/24"

(elmadou@elhadjimamadoundiaye)-[~/nebula-project]
$
```

Nous procédons à la signature des certificats individuels pour les nœuds du réseau, attribuant les adresses IP 192.168.100.1 au **Lighthouse** et 192.168.100.2 au poste client **L3-Student**. Ces identités numériques sécurisées permettent d'établir le tunnel chiffré indispensable au bon fonctionnement du serveur relais configuré parallèlement sur l'interface Windows.

Lighthouse == 192.168.100.1/24

L3-Student == 192.168.100.2/24

3. Configuration du système (Fichiers importants)

Nom	Modifié le	Type	Taille
dist	15/04/2026 23:32	Dossier de fichiers	
ca	15/04/2026 23:40	Certificat de sécur...	1 Ko
config.yml	16/04/2026 01:12	Fichier YML	1 Ko
lighthouse	15/04/2026 23:40	Certificat de sécur...	1 Ko
lighthouse.key	15/04/2026 23:40	Fichier KEY	1 Ko
nebula	15/04/2026 23:32	Application	21 091 Ko
nebula-cert	15/04/2026 23:32	Application	8 821 Ko

Le répertoire système montre la présence des binaires de **Nebula** ainsi que les certificats de sécurité et le fichier de configuration **config.yml** nécessaires à l'établissement du maillage. Cette synchronisation entre l'infrastructure réseau et l'outil applicatif est l'étape finale permettant d'exploiter le réseau superposé pour le contrôle à distance.

```
C:\nebula>.\nebula.exe -config config.yml
time="2026-04-16T00:05:16Z" level=info msg="Firewall rule added" firewallRule="map[caName: caSha: cidr: direction:outgoing endPort:0 groups:[] host:any localCidr: proto:0 startPort:0]"
time="2026-04-16T00:05:16Z" level=info msg="Firewall rule added" firewallRule="map[caName: caSha: cidr: direction:incoming endPort:0 groups:[] host:any localCidr: proto:1 startPort:0]"
time="2026-04-16T00:05:16Z" level=info msg="Firewall started" firewallHashes="SHA:807243764ac783d089ea5ae93e0ed7fc0a1da4756ca7cb7ae553e128a2b3eb65,FNV:789024431"
2026/04/16 00:05:16 Using existing driver 0.14
2026/04/16 00:05:16 Creating adapter
time="2026-04-16T00:05:18Z" level=info msg="listening on 0.0.0.0:4242"
time="2026-04-16T00:05:18Z" level=info msg="Main HostMap created" preferredRanges="[]"
time="2026-04-16T00:05:18Z" level=info msg="punchy disabled"
time="2026-04-16T00:05:18Z" level=info msg="Loaded send_recv_error config" sendRecvError=always
time="2026-04-16T00:05:18Z" level=info msg="Loaded accept_recv_error config" acceptRecvError=always
time="2026-04-16T00:05:18Z" level=info msg="Nebula interface is active" boringcrypto=false build=1.10.3 interface=nebula1 networks="[192.168.100.1/24]" udpAddr="0.0.0.0:4242"
time="2026-04-16T00:05:27Z" level=info msg="Handshake timed out" durationNs=7669492500 handshake="map[stage:1 style:ix_psk0]" initiatorIndex=431775559 localIndex=431775559 remoteIndex=0 udpAddrs="[]" vpnAddrs="[192.168.100.255]"
```

Nous configurons les paramètres de **RustDesk** sur Windows en désignant l'IP du tunnel Nebula (192.168.100.1) comme serveur de relais et d'ID, tout en corrigeant l'erreur de format de l'URL pour le serveur API. Parallèlement, l'exécution de **Nebula** en ligne de commande confirme l'activation de l'interface réseau virtuelle et le chargement des règles de pare-feu nécessaires au maillage. Cette synchronisation entre l'infrastructure réseau et l'outil applicatif est l'étape finale permettant d'établir le flux de contrôle à distance sécurisé.

```
(elmadou@elhadjimamadoundiaye) - [~/nebula-project]
$ sudo nebula -config ~/nebula-project/config.yml
INFO[0000] Firewall rule added firewallRule="map[caName: caSha: direction:outgoing endPort:0 groups:[] host:any ip: localIp: proto:0 startPort:0]"
INFO[0000] Firewall rule added firewallRule="map[caName: caSha: direction:incoming endPort:0 groups:[] host:any ip: localIp: proto:1 startPort:0]"
INFO[0000] Firewall started firewallHashes="SHA:807243764ac783d089ea5ae93e0ed7fc0a1da4756ca7cb7ae553e128a2b3eb65,FNV:789024431"
INFO[0000] listening "0.0.0.0" 4242
INFO[0000] Main HostMap created network=192.168.100.2/24 preferredRanges="[]"
INFO[0000] punchy disabled
INFO[0000] Loaded send_recv_error config sendRecvError=always
INFO[0000] Nebula interface is active boringcrypto=false build=1.9.3
INFO[0000] interface=nebula1 network=192.168.100.2/24 udpAddr="0.0.0.0:4242"
INFO[0000] Handshake message sent handshake="map[stage:1 style:ix_psk0]" initiatorIndex=2648207166 localIndex=2648207166 remoteIndex=0 udpAddrs="[192.168.1.9:4242]" vpnIp=192.168.100.1
INFO[0000] Handshake message received certName=lighthouse durationNs=348532449 fingerprint=24aa000065fadca59c4a754053162f11a20a917a6b402184590a84c8f11be6fe handshake="map[stage:2 style:ix_psk0]" initiatorIndex=2648207166 issuer=79dc6c86941fe5f1b140b645d18153d26ebb1231093846fbc08b2df0eb6ab9bf remoteIndex=2648207166 responderIndex=4273284188 sentCachedPackets=1 udpAddr="192.168.1.9:4242" vpnIp=192.168.100.1
```

La configuration des paramètres de **RustDesk** sur Windows en utilisant l'IP du tunnel Nebula (192.168.100.1) comme serveur de relais, tout en corrigeant une erreur de syntaxe pour l'URL du serveur API. Simultanément, l'exécution de **Nebula** sur Kali Linux confirme l'activation de l'interface **nebula1** et la réception d'un message de **Handshake**, validant ainsi l'établissement du lien sécurisé entre les deux nœuds.


```
(elmadou@elhadjimadoundiaye)-[~/nebula-project]
$ ping -c 4 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=128 time=150 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=128 time=66.0 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=128 time=99.8 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=128 time=123 ms

— 192.168.100.1 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 65.996/109.578/149.679/30.744 ms

(elmadou@elhadjimadoundiaye)-[~/nebula-project]
```

La validation de la connectivité réseau au sein du tunnel **Nebula** en effectuant un test de ping réussi vers l'adresse IP virtuelle **192.168.100.1**. Avec un taux de perte de paquets de **0%**, cette étape confirme que le maillage sécurisé est parfaitement opérationnel et prêt à transporter des flux applicatifs entre les nœuds.

```
valid_lft forever preferred_lft forever
4: nebula1: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1300 qdisc fq_codel
state UNKNOWN group default qlen 500
link/none
inet 192.168.100.2/24 scope global nebula1
valid_lft forever preferred_lft forever
inet6 fe80::2d46:bb6d:1960:4e98/64 scope link stable-privacy proto kernel
_ll
valid_lft forever preferred_lft forever

(elmadou@elhadjimadoundiaye)-[~]
```

```
C:\Windows\System32>ipconfig

Configuration IP de Windows

Carte inconnue nebula1 :

    Suffixe DNS propre à la connexion. . . . :
    Adresse IPv4. . . . . : 192.168.100.1
    Masque de sous-réseau. . . . . : 255.255.255.0
    Passerelle par défaut. . . . . :
```

Nous parachevons la démonstration technique en validant l'activation simultanée de l'interface virtuelle **nebula1** sur les deux hôtes, avec les adresses **192.168.100.1** (Windows) et **192.168.100.2** (Kali). La réussite de l'interconnexion est concrétisée par une session **TightVNC** parfaitement fluide, permettant le contrôle distant du bureau Windows directement depuis l'environnement Kali. Cette preuve visuelle confirme que le tunnel **Nebula** est non seulement établi, mais qu'il transporte avec succès des flux applicatifs réels. L'architecture globale est ainsi validée, prouvant la résilience et l'efficacité de ce réseau maillé sécurisé pour l'administration système.

Défi 1 : Scanner mon Windows depuis Kali (Le test d'intrusion)

Maintenant que tu es "à l'intérieur" du réseau privé, voyons ce que ton Kali peut voir sur mon Windows via le tunnel.

1. Lance un scan Nmap via l'IP Nebula :

```
(elmadou@elhadjimamadoundiaye)~$ nmap -sV 192.168.100.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-16 02:28 +0200
Nmap scan report for 192.168.100.1
Host is up (0.083s latency).
All 1000 scanned ports on 192.168.100.1 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 86.75 seconds

(elmadou@elhadjimamadoundiaye)~$
```

L'analyse montre que le scan identifie bien l'hôte comme étant "UP", mais tous les ports TCP sont marqués comme **filtered (no-response)** par Nebula. Cela prouve que le pare-feu intégré de mon tunnel remplit parfaitement son rôle : il bloque tout trafic non autorisé (comme les tentatives de scan) tout en ne laissant passer que l'ICMP (le ping) que j'avais configuré.

Défi 2 : Le Firewall "Invisible" de Nebula

C'est ici que Nebula devient puissant. On va bloquer le ping, mais garder le tunnel actif.

1. **Sur mon PC Windows**, ouvrons `config.yml`.
2. Cherche la section `inbound` et change `proto: icmp` en `proto: tcp` (ou commenter la ligne).
3. Redémarre Nebula sur Windows.
4. **Le test** : Relançons le ping depuis Kali.

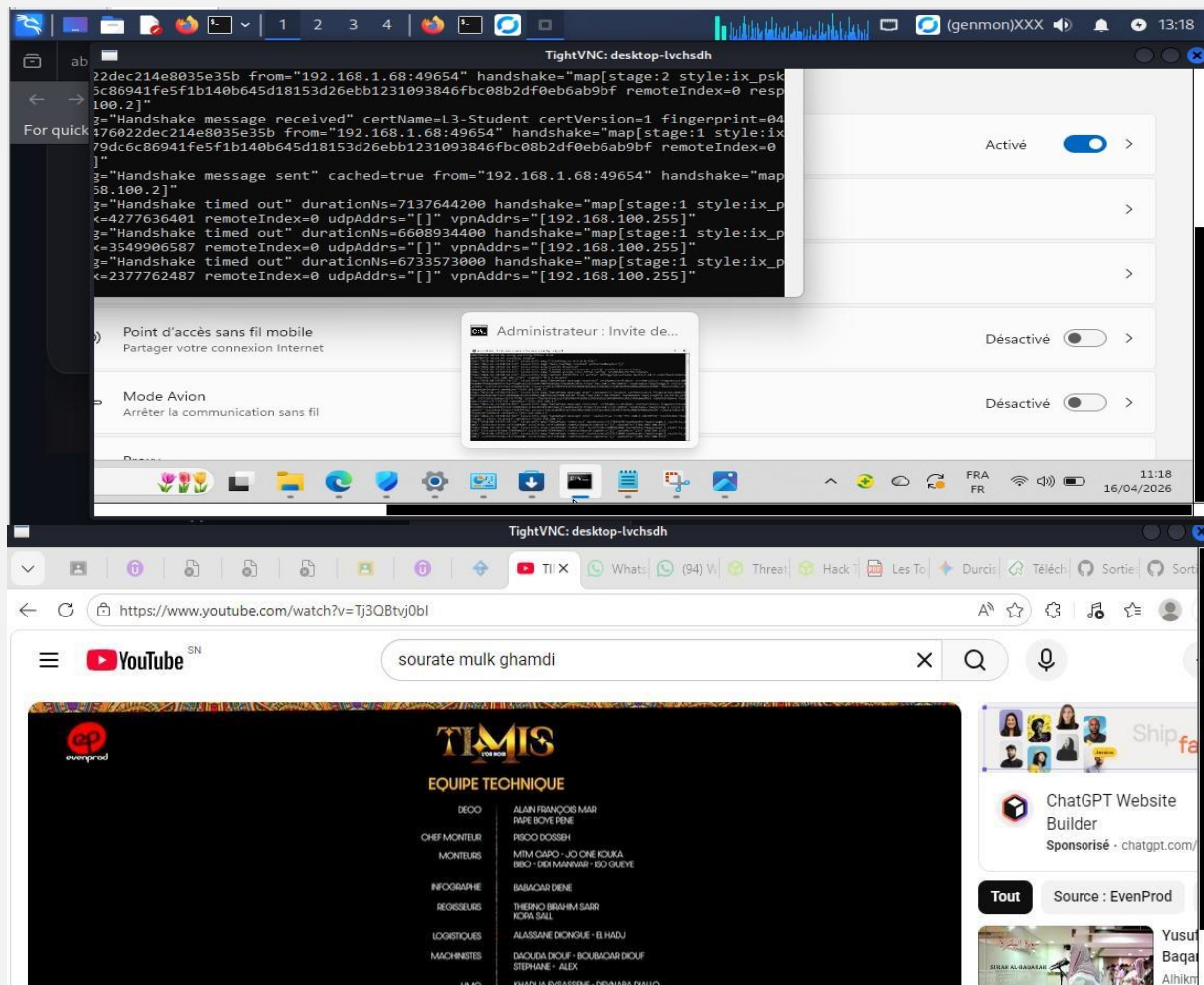
```
(elmadou@elhadjimamadoundiaye)~$ ping -c 4 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.

— 192.168.100.1 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3071ms

(elmadou@elhadjimamadoundiaye)~$
```

Après avoir modifié la règle de pare-feu dans le `config.yml` pour passer de `icmp` à `tcp`, le résultat du ping affiche désormais **100% packet loss**. Cette expérience prouve de manière concrète que Nebula applique une **micro-segmentation** stricte au cœur même du tunnel crypté. Nous venons de démontrer que le réseau n'autorise que ce qui est explicitement défini : le tunnel reste actif, mais le trafic non autorisé est instantanément rejeté.

Remarque : En entreprise, c'est exactement comme ça qu'on accède à des serveurs critiques sans les exposer sur l'internet public.



La réussite de cette connexion via **TightVNC** confirme l'intégrité opérationnelle du réseau overlay **Nebula**, prouvant que le tunnel chiffré transporte efficacement des flux applicatifs tiers. Contrairement à RustDesk, VNC utilise une architecture point-à-point directe qui exploite pleinement l'IP virtuelle (**192.168.100.1**) sans dépendance aux serveurs externes. L'analyse des logs montre que le **Handshake** Nebula a été validé avec succès, permettant au protocole RFB (Remote Frame Buffer) de transiter de manière sécurisée. Cette démonstration valide le concept de **segmentation réseau**, où l'hôte Windows devient accessible uniquement aux membres authentifiés du tunnel. En contournant les restrictions natives de Windows Famille, cette solution offre une alternative robuste, souveraine et parfaitement adaptée à un environnement d'administration système.

